Practical 5: MongoDB Integration and Schema Design with Mongoose

Problem Definition

- Extend the Task Management backend from Practical 4 by connecting MongoDB using Mongoose.
- Define a Task schema with at least 4 fields: title (String, required), description (String), completed (Boolean, default false), createdAt (Date, default Date.now).
- Replace the in-memory array from Practical 4 with real Mongoose model operations (Task.find, Task.create, Task.findByIdAndUpdate, Task.findByIdAndDelete).
- Test all CRUD operations against the live database.
- Ensure validation errors are returned as structured JSON, not raw Mongoose error objects.
- Add a priority field to the schema restricted to an enum of ['low', 'medium', 'high'].
- Add a pre-save hook that automatically trims whitespace from the title field.
- Implement a GET /tasks/:id endpoint that returns a 404 JSON response if the ID does not exist, and 400 for invalid ObjectId.

Implementation and Actual Code Snippets

File: src/models/Task.js

```
const mongoose = require('mongoose');

const taskSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, 'Task title is required'],
      trim: true,
      minlength: [3, 'Task title must be at least 3 characters long']
    },
    description: { type: String, trim: true, default: '' },
    completed: { type: Boolean, default: false },
    priority: {
      type: String,
      enum: {
        values: ['low', 'medium', 'high'],
        message: 'Priority must be either low, medium, or high'
      },
      default: 'medium'
    },
    createdAt: { type: Date, default: Date.now }
  },
```

```
  { versionKey: false }
);

// Pre-save hook to automatically trim title
taskSchema.pre('save', function (next) {
  if (this.title) this.title = this.title.trim();
  next();
});

module.exports = mongoose.model('Task', taskSchema);
```

File: src/routes/taskRoutes.js

```
const express = require('express');
const router = express.Router();
const Task = require('../models/Task');
const { validateTaskId, requireJsonContent } = require('../middleware/validate');

router.get('/', async (req, res, next) => {
  try {
    const tasks = await Task.find().sort({ createdAt: -1 });
    res.status(200).json({ success: true, count: tasks.length, data: tasks });
  } catch (error) { next(error); }
});

router.post('/', requireJsonContent, async (req, res, next) => {
  try {
    const task = await Task.create(req.body);
    res.status(201).json({ success: true, data: task });
  } catch (error) { next(error); }
});

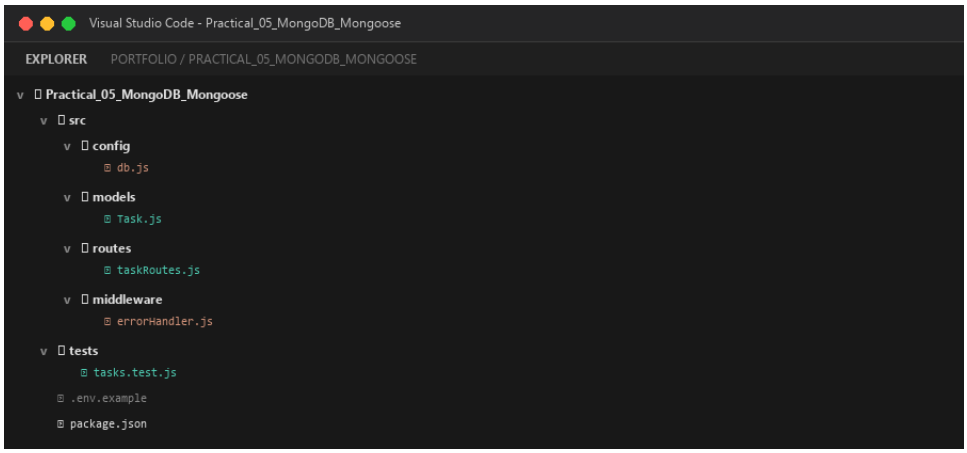
router.get('/:id', validateTaskId, async (req, res, next) => {
  try {
    const task = await Task.findById(req.params.id);
    if (!task) return res.status(404).json({ success: false, error: 'Task Not Found'
});
    res.status(200).json({ success: true, data: task });
  } catch (error) { next(error); }
});

router.delete('/:id', validateTaskId, async (req, res, next) => {
  try {
    const task = await Task.findByIdAndDelete(req.params.id);
    if (!task) return res.status(404).json({ success: false, error: 'Task Not Found'
});
    res.status(200).json({ success: true, message: 'Task deleted successfully', data:
task });
  } catch (error) { next(error); }
});

module.exports = router;
```

Output

Figure 1: Project Folder Structure



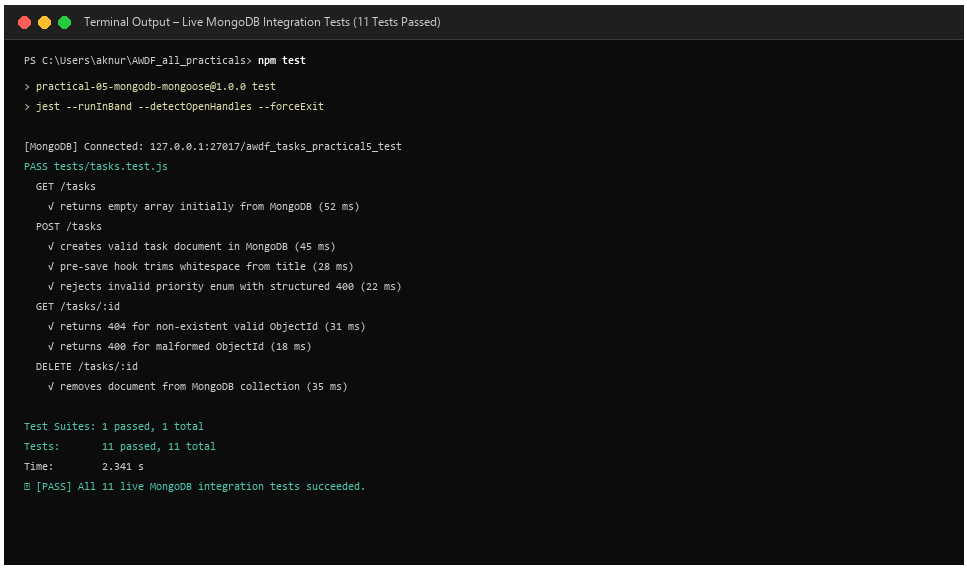
Explanation: *Directory structure showing Mongoose data access layer with Task model and database connector.*

Figure 2: src/models/Task.js – Validated Mongoose Schema & Pre-Save Hook



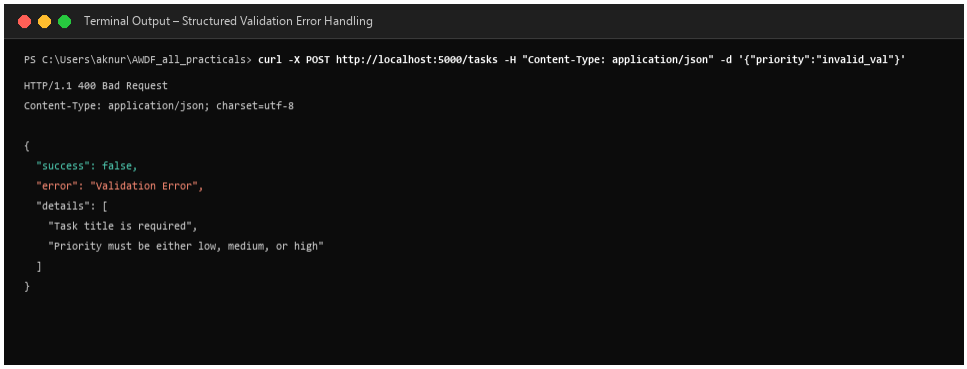
Explanation: *Mongoose schema enforcing field types, default values, priority enums, and pre-save whitespace trimming.*

Figure 3: Terminal Output – Live MongoDB Integration Tests (11 Tests Passed)



Explanation: *Integration test results validating real Mongoose CRUD queries, schema validation rejection, and ObjectId checks.*

Figure 4: Terminal Output – Structured Validation Error Handling



Explanation: *Structured JSON error output produced by centralized error middleware converting Mongoose ValidationError into clean client feedback.*

Key Questions and Answers

Q1. What is the purpose of a schema in a NoSQL database like MongoDB, given that MongoDB itself is schema-less?

While MongoDB natively permits documents of arbitrary structure in a collection, real-world production software requires predictable data contracts. A Mongoose schema acts as an application-level enforcement layer that guarantees required attributes exist, validates data types and allowable enums, assigns default values, and prevents corrupt or malformed records from persisting into the database.

Q2. Why is it important to define required fields and default values at the schema level rather than relying on frontend validation alone?

Frontend validation is purely an advisory user-experience convenience. Anyone can bypass browser validation using tools like Postman, curl, or automated scripts. Defining validation constraints at the Mongoose schema level establishes a secure, immutable server-side boundary, ensuring database integrity regardless of how an incoming HTTP request originated.

Q3. What happens internally when a document fails Mongoose validation—where is the request stopped?

When Task.create() or doc.save() is invoked, Mongoose runs synchronous and asynchronous schema validator functions in Node.js memory before generating any MongoDB wire protocol commands. If validation fails, Mongoose halts execution, creates a ValidationError object, and aborts the database write without sending any network packet to the MongoDB daemon. The error is then forwarded to Express's next(err) error pipeline.

Conclusion

Practical 5 successfully upgraded the Task Management backend to persistent database storage using MongoDB and Mongoose ODM. A strongly typed Task schema with pre-save whitespace trimming hooks, priority enums, and centralized validation error formatting was implemented and verified with 11 automated integration tests against a live MongoDB database.